Практична робота № 3

Тема: Налаштування захищеного каналу mTLS та протокол SMTP

Мета роботи

- Опанувати принципи функціонування інфраструктури відкритих ключів (PKI) та генерації сертифікатів стандарту X.509 (Root CA, серверні та клієнтські сертифікати).
- Набути практичних навичок конфігурації захищених TCP-з'єднань на базі пакета `crypto/tls` із забезпеченням двосторонньої автентифікації (mTLS) та верифікації сертифікатів.
- Опанувати роботу з протоколом електронної пошти SMTP за допомогою стандартного пакета `net/smtp` та механізму захисту STARTTLS.
- Навчитися формувати багатокомпонентні повідомлення формату MIME (HTML-розмітка та вкладення файлів) і тестувати доставку через локальні емулятори (Mailpit, MailHog) або зовнішні поштові сервіси.

Технічне забезпечення

- Операційна система: Windows, macOS або Linux.
- Середовище розробки: Visual Studio Code (з офіційним розширенням Go) або JetBrains GoLand.
- Компілятор: Go SDK версії 1.22 або новішої.
- Термінал / CLI: bash, zsh або PowerShell.
- Система контролю версій: Git.
- Інструменти криптографії: утиліта `openssl` (або скрипт генерації ключів).
- Інструменти тестування SMTP: локальний емулятор Mailpit / MailHog (бінарник або Docker-контейнер) або активний поштовий акаунт (наприклад, Gmail з паролем застосунку).

Короткі теоретичні відомості

1. Інфраструктура відкритих ключів (PKI) та взаємна автентифікація (mTLS)

У стандартному TLS (односторонньому) тільки клієнт перевіряє автентичність сервера. Сервер надає свій сертифікат, а клієнт перевіряє його дійсність за допомогою списку довірених центрів сертифікації (Root CA).

При **mTLS (Mutual TLS)** сервер і клієнт перевіряють цифрові сертифікати один одного:

1. Клієнт перевіряє сертифікат сервера (захист від атак Man-in-the-Middle).
2. Сервер вимагає та перевіряє клієнтський сертифікат під час TLS-рукостискання (автентифікація клієнта на транспортному рівні без логінів і паролів).

Для тестування mTLS створюється локальний ланцюжок довіри:

- **Root CA:** кореневий сертифікат (`ca.crt`) і закритий ключ (`ca.key`);
- **Сервер:** сертифікат (`server.crt`) і ключ (`server.key`), підписані Root CA;
- **Клієнт:** сертифікат (`client.crt`) і ключ (`client.key`), підписані тим самим Root CA.

Генерація сертифікатів за допомогою утиліти `openssl`:

```
# 1. Створення Root CA
openssl req -x509 -newkey rsa:2048 -nodes -keyout ca.key -out ca.crt -days 365 -subj "/CN=MyRootCA" ①

# 2. Сертифікат сервера (із SAN localhost/127.0.0.1)
openssl req -newkey rsa:2048 -nodes -keyout server.key -out server.csr -subj "/CN=localhost" ②
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt -days 365 \
  -extfile <(echo "subjectAltName=DNS:localhost,IP:127.0.0.1") ③

# 3. Сертифікат клієнта
openssl req -newkey rsa:2048 -nodes -keyout client.key -out client.csr -subj "/CN=dispatcher-agent-01" ④
openssl x509 -req -in client.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out client.crt -days 365 ⑤
```

① Генерація самопідписаного кореневого сертифіката (CA).

② Створення запиту на підпис сертифіката (CSR) сервера.

③ Підпис сертифіката сервера за допомогою Root CA з обов'язковим додаванням SAN (Subject Alternative Name).

④ Генерація закритого ключа та запиту клієнта з унікальним ідентифікатором у Common

Name.

⑤ Підпис сертифіката клієнта тим самим кореневим центром сертифікації.

---

2. Налаштування mTLS у Go (`crypto/tls`)

Пакет `crypto/tls` надає засоби створення як захищених серверів (`tls.Listen`), так і клієнтів (`tls.Dial`).

Ключова конфігурація сервера для виклику mTLS:

```
// Завантаження сертифіката сервера
cert, err := tls.LoadX509KeyPair("server.crt", "server.key") ①

// Завантаження довіреного Root CA для перевірки клієнтів
caCert, err := os.ReadFile("ca.crt")
caCertPool := x509.NewCertPool()
caCertPool.AppendCertsFromPEM(caCert) ②

serverTLSConfig := &tls.Config{
    Certificates: []tls.Certificate{cert},
    ClientAuth:   tls.RequireAndVerifyClientCert, ③
    ClientCAs:    caCertPool,                     ④
    MinVersion:   tls.VersionTLS13,
}

listener, err := tls.Listen("tcp", ":8443", serverTLSConfig) ⑤
```

① Парсинг пари серверного сертифіката та закритого ключа у пам’ять.

② Створення пулу кореневих сертифікатів для верифікації клієнтів.

③ Критична опція для mTLS: сервер зобов’язує клієнта надати сертифікат і перевіряє його дійсність.

④ Вказівка довіреного CA, яким повинен бути підписаний клієнтський сертифікат.

⑤ Відкриття захищеного TLS-сокета поверх стандартного протоколу TCP.

Клієнтська конфігурація для проходження перевірки:

```
clientCert, err := tls.LoadX509KeyPair("client.crt", "client.key") ①
caCert, err := os.ReadFile("ca.crt")
caCertPool := x509.NewCertPool()
caCertPool.AppendCertsFromPEM(caCert)

clientTLSConfig := &tls.Config{
    Certificates: []tls.Certificate{clientCert}, ②
    RootCAs:      caCertPool,                     ③
}
```

```
10
11 conn, err := tls.Dial("tcp", "127.0.0.1:8443", clientTLSConfig) ④
```

① Завантаження сертифіката та ключа клієнта.

② Надання клієнтського сертифіката серверу під час рукостискання.

③ Перевірка серверного сертифіката через спільний Root CA.

④ Встановлення взаємно автентифікованого TLS-з'єднання.

---

3. Протокол SMTP та відправка пошти (`net/smtp`)

Протокол SMTP (Simple Mail Transfer Agent) передає текстові команди завершені `\r\n`. Стандартний пакет `net/smtp` містить функцію `smtp.SendMail` для швидкого надсилання листів:

```
1 auth := smtp.PlainAuth("", "user@example.com", "password", "smtp.example.com") ①
2 // Для локальних емуляторів без авторизації (Mailpit/MailHog) auth передається як
  nil
3
4 to := []string{"admin@example.com"}
5 msg := []byte("Subject: Тестове сповіщення\r\n\r\nТекст листа") ②
6
7 err := smtp.SendMail("localhost:1025", nil, "server@example.com", to, msg) ③
```

① Налаштування PLAIN-автентифікації для зовнішнього поштового сервера.

② Формування базового поштового повідомлення з обов'язковим розділенням заголовків і тіла двома переносами рядка.

③ Відправка листа на вказаний поштовий сервер (порт 1025 для Mailpit/MailHog).

---

4. Формат повідомлень MIME: HTML та вкладення (Attachments)

Стандарт RFC 2045 (MIME) описує структуру електронних листів із підтримкою форматованого тексту та бінарних вкладень:

- Заголовок `Content-Type: multipart/mixed; boundary=⋯` розділяє повідомлення на логічні частини довільним унікальним текстовим розділювачем (`boundary`).
- Тіло кожної секції починається з власних MIME-заголовків (`Content-Type: text/html; charset=UTF-8`, `Content-Disposition: attachment; filename=⋯`) та відокремлюється порожнім рядком `\r\n\r\n`.
- Бінарні файли чи звіти кодуються в текстовий формат **Base64** за допомогою пакета `encoding/base64`.

Постановка задачі

У корпоративних розподілених мережах критично важливі сервіси (наприклад, банківські транзакційні шлюзи або сервери моніторингу електропідстанцій) не повинні бути доступними через відкритий інтернет для анонімних користувачів. Згідно з концепцією **Zero Trust**, кожна взаємодія між підсистемами повинна проходити сувору криптографічну взаємну ідентифікацію на транспортному рівні (mTLS).

У разі реєстрації критичного збою підключений оператор або датчик надсилає сигнал тривоги. Шлюз перевіряє правомірність клієнта, фіксує його ідентифікатор із сертифіката та негайно формує детальне сповіщення відповідальній службі через поштовий протокол SMTP.

Необхідно розробити консольний **шлюз критичних сповіщень та аудиту (Secure Alert Gateway)**, що складається з:

1. **mTLS TCP-сервера**, який приймає з'єднання виключно від клієнтів з валідними сертифікатами спільного CA;
2. **mTLS-клієнта**, який передає термінове повідомлення про подію або аварію;
3. **SMTP-сервісу**, який формує структурований поштовий звіт та відправляє його черговому інженеру на локальний тестовий емулятор або реальний поштовий сервер.

Завдання

Практична робота складається з трьох послідовних рівнів. Кожен наступний рівень базується на попередньому та розширює функціональність розроблюваного рішення.

Рівень 1. Базовий: Генерація PKI та захищений mTLS-шлюз

1. **Ініціалізація проєкту:**
   - Створити окрему робочу директорію для практичної роботи.
   - Ініціалізувати Go-модуль за допомогою команди `go mod init <назва_модуля>`.
2. **Генерація інфраструктури сертифікатів (PKI):**
   - Створити окрему папку `certs/`.
   - Згенерувати кореневий сертифікат локального центру сертифікації (`ca.crt`, `ca.key`).
   - Згенерувати закритий ключ і сертифікат сервера (`server.key`, `server.crt`), підписані створеним Root CA, вказавши у розширенні SAN адресу `localhost` або `127.0.0.1`.
   - Згенерувати закритий ключ і сертифікат клієнта (`client.key`, `client.crt`), підписані тим самим Root CA, з атрибутом `CN=dispatcher-agent-01`.

- Зберегти команди генерації у скрипті `generate_certs.sh` (для Linux/macOS) або `generate_certs.bat` / `generate_certs.ps1` (для Windows).

3. **Реалізація mTLS-сервера:**
   - У файлі `server/main.go` створити TCP-сервер на базі пакета `crypto/tls` (порт `:8443`).
   - Налаштувати `tls.Config`: завантажити серверні ключі, завантажити `ca.crt` у `ClientCAs` та встановити політику автентифікації `tls.RequireAndVerifyClientCert`.
   - Реалізувати прийом текстового повідомлення від підключеного клієнта, виведення тексту у консоль сервера та відправку назад підтвердження: `[SERVER]: ALERT RECEIVED AND VERIFIED`.
4. **Створення авторизованого mTLS-клієнта:**
   - У файлі `client/main.go` реалізувати клієнтське підключення через `tls.Dial`.
   - Налаштувати `tls.Config`: підключити клієнтський сертифікат (`client.crt`, `client.key`) та довірений Root CA (`ca.crt`).
   - Надіслати тестове повідомлення про інцидент: `CRITICAL_ERROR: Database connection timeout on node-04`.
5. **Тестування відхилення неавторизованого доступу:**
   - Спробувати підключитися до сервера клієнтом без сертифіката (наприклад, через команду `openssl s_client -connect localhost:8443` без прапорця `-cert` або через утиліту `curl https://localhost:8443 -k`).
   - Переконатися, що сервер розриває з'єднання під час TLS-рукостискання з помилкою відсутності або недійсності сертифіката.

Рівень 2. Ключовий: Автоматична відправка email-сповіщень через SMTP

1. **Збереження функціональності Рівня 1.**
2. **Підготовка середовища SMTP-сервера (на вибір студента):**
   - **Варіант А (Локальний емулятор — рекомендовано для розробки):** завантажити та запустити утиліту Mailpit (`./mailpit`) або запустити контейнер (`docker run -d -p 1025:1025 -p 8025:8025 axllent/mailpit`). Порт 1025 — SMTP, порт 8025 — вебінтерфейс. Або скористатися емулятором MailHog.
   - **Варіант Б (Зовнішній поштовий шлюз):** налаштувати SMTP-акаунт (Gmail через «Паролі додатків», Ukr.net або корпоративний сервер) з авторизацією через `smtp.PlainAuth`.
3. **Розробка модуля відправки пошти (Email Service):**
   - Створити окремий пакет або модуль для роботи з поштою за допомогою `net/smtp`.
   - Реалізувати параметризовану функцію надсилання листа, яка приймає адресу отримувача, тему (`Subject`) та текст повідомлення.

- Реалізувати коректне формування стандартних поштових заголовків `From:`, `To:`, `Subject:`.

4. **Інтеграція обробника подій на сервері:**
   - Після успішного прийому mTLS-повідомлення про інцидент сервер автоматично викликає функцію надсилання email-сповіщення черговому інженеру (`duty-engineer@internal.net`).
   - У тексті листа передати отримане повідомлення про інцидент та дату/час реєстрації події.
   - Перевірити отримання листа у вебінтерфейсі Mailpit/MailHog або у поштовій скриньці.

Рівень 3. Просунутий: Аудит Zero Trust, MIME-форматування та вкладення файлів

1. **Збереження функціональності попередніх рівнів.**
2. **Аудит суб'єкта сертифіката (Zero Trust Identity):**
   - Отримати доступ до стану захищеного з'єднання через метод `conn.(*tls.Conn).ConnectionState()`.
   - Витягти сертифікат клієнта зі зрізу `PeerCertificates`.
   - Прочитати ідентифікатор клієнта: `Subject.CommonName` (CN), серійний номер сертифіката та термін дії.
   - Додати інформацію про автора запиту до системного логу сервера та у текст сповіщення (наприклад: **«Запит надійшов від верифікованого пристрою [CN=dispatcher-agent-01]»**).
3. **Формування багатокомпонентного листа MIME (HTML + Attachment):**
   - Зібрати лист відповідно до специфікації MIME з розділювачем частин (`boundary`):
     - **Секція 1 (HTML):** візуально оформлене тіло листа з таблицею або блоком статусу (червоний індикатор тривоги, точний час, джерело події, ідентифікатор із сертифіката);
     - **Секція 2 (Вкладений файл):** звіт аудиту події `incident_report.log` або технічний дамп стану системи, закодований за алгоритмом Base64 (`encoding/base64`).
   - Переконатися, що поштовий клієнт (або вебінтерфейс Mailpit) коректно рендерить HTML-верстку та дозволяє завантажити й переглянути прикріплений файл вкладення.

Приклад очікуваного виведення програми

Консоль mTLS-сервера:

```
=== Захищений шлюз критичних сповіщень (mTLS: 8443, SMTP Target: 1025) ===
[СЕРВЕР] Пул CA завантажено: ca.crt (1 сертифікат)
[СЕРВЕР] Слухач mTLS відкрито на порту :8443
[ПІДКЛЮЧЕННЯ] Встановлено TLS-з'єднання від 127.0.0.1:58432
[АУДИТ ZERO TRUST] Клієнт верифікований: CN=dispatcher-agent-01, Серійний
номер=492048...
[ПОВІДОМЛЕННЯ] Отримано інцидент: CRITICAL_ERROR: Database connection timeout on node-
04
[SMTP] Формування MIME-повідомлення (HTML + вкладення incident_report.log)...
[SMTP] Надсилання email на duty-engineer@internal.net через 127.0.0.1:1025...
[SMTP] □ Лист про аварію успішно доставлено до поштового шлюзу!
[ВІДПОВІДЬ] Клієнту надіслано статус: [SERVER]: ALERT PROCESSED AND DISPATCHED
```

Консоль авторизованого mTLS-клієнта:

```
=== Агент диспетчеризації (mTLS Client) ===
[КЛІЄНТ] Завантажено сертифікат клієнта: client.crt (CN=dispatcher-agent-01)
[КЛІЄНТ] Підключення до шлюзу 127.0.0.1:8443...
[КЛІЄНТ] mTLS-рукостискання успішне. Використано протокол TLS 1.3
[КЛІЄНТ] Відправка алерту: CRITICAL_ERROR: Database connection timeout on node-04
[КЛІЄНТ] Відповідь від сервера: [SERVER]: ALERT PROCESSED AND DISPATCHED
[КЛІЄНТ] Сесію успішно завершено.
```

Спроба неавторизованого підключення (без клієнтського сертифіката):

```
1 curl -k https://localhost:8443
```

```
curl: (35) error:14094412:SSL routines:ssl3_read_bytes:sslv3 alert bad certificate
```

(У консолі сервера фіксується помилка: `tls: client didn□t provide a certificate` або `bad certificate`, з'єднання безпечно відхиляється).

Контрольні питання

1. Чим відрізняється стандартний односторонній протокол TLS від взаємного TLS (mTLS) під час рукостискання?
2. Яку роль відіграє параметр `tls.RequireAndVerifyClientCert` у структурі `tls.Config` сервера Go?
3. Чому для внутрішньої взаємодії мікросервісів у моделі Zero Trust часто використовують саме mTLS замість традиційної передачі паролів чи API-ключів у заголовках?
4. Яке призначення поля SAN (Subject Alternative Name) у сертифікатах X.509 і чому сучасні браузери та Go ігнорують поле Common Name для перевірки доменів?
5. Чим відрізняється робота поштового протоколу через механізм STARTTLS (порт 587) від SMTP over SSL/TLS (порт 465)?
6. Як влаштований багатокомпонентний лист формату MIME (Multipart) та яку функцію виконує параметр `boundary`?
7. Чому бінарні файли вкладень у поштових листах необхідно кодувати за допомогою алгоритму Base64?
8. Які переваги дає використання локальних SMTP-емуляторів (Mailpit, MailHog) під час розробки та автоматизованого тестування ПЗ?

Критерії оцінювання

Оцінювання здійснюється за шкалою повноти реалізації рівнів практичної роботи:

- **«Задовільно» (Рівень 1):** реалізовано базовий прототип, створено структуру проєкту, програма успішно компілюється та виконує базові сценарії.
- **«Добре» (Рівень 2):** виконано базовий рівень та реалізовано ключову функціональність заняття, забезпечено надійність і безпеку виконання коду.
- **«Відмінно» (Рівень 3):** повністю завершено всі три рівні завдання з дотриманням кращих інженерних практик, чистоти архітектури та оптимізації.
- **Усний захист:** обов'язкове пояснення логіки коду та впевнені відповіді на контрольні питання викладача.

Вимоги до звітності

1. Опублікувати вихідний код у Git-репозиторії.
2. Оформити файл `README.md` з описом проєкту, інструкцією запуску та обов'язковими скриншотами роботи програми (включаючи демонстрацію відсутності помилок).
3. Продемонструвати роботу програми викладачеві та захистити результати.